Software Requirements Specification (SRS)

VaultKeep – Digital Warranty & Service Book

First Year Engineering Project

Date: July 23, 2026

Version: 1.0

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for VaultKeep, a web-based Digital Warranty and Service Book. VaultKeep allows users to securely store purchase bills, warranty cards, service records, and product information in a centralized digital vault, while proactively reminding them before warranty expiry and scheduled service due dates.

1.2 Scope

VaultKeep is a web application that provides the following functionalities:

- Secure user registration and login
- Product registration and management
- Upload and storage of bills, warranty cards, and product images
- Service history logging for each product
- Automated warranty and service due reminders via email
- QR code generation for quick access to product records
- Downloadable reports (PDF) of products and warranties
- Platform administration (user management, monitoring, reporting)

The platform aims to eliminate the loss of paper warranty cards and bills, and to help users track service schedules effortlessly.

1.3 Objectives

- Provide a secure digital repository for purchase and warranty documents.
- Notify users before warranty expiry and service due dates.
- Simplify product and service record management through a single dashboard.
- Enable quick retrieval of product details using QR codes.
- Generate simple reports for personal record-keeping.
- Provide a secure, responsive, and user-friendly platform.

1.4 Intended Audience

- Individual consumers who purchase electronics, appliances, and other warranted products
- Small retailers/service centers issuing warranty documentation
- Platform Administrator
- Developers and Project Evaluators

1.5 Definitions

Term	Description
User	A registered individual who owns products stored in VaultKeep.
Product	An item registered by a user, associated with a bill and warranty.
Warranty	The manufacturer/seller assurance period linked to a product.
Document	A scanned/uploaded bill, warranty card, or product image.
Service Record	A logged entry of a service/repair event for a product.
Admin	Platform operator who manages users and monitors system activity.
Notification	System-generated alert (email) about an upcoming due date.

2. Overall Description

2.1 Product Perspective

VaultKeep is a standalone web application that acts as a personal digital vault for warranty and service-related documentation. It is implemented using a PHP backend with a MySQL database, hosted via XAMPP for development.

2.2 Product Functions

- Register and authenticate users securely.

- Allow users to add products along with bills, warranty cards, and images.
- Track service history for each product.
- Automatically monitor warranty expiry and service due dates, and send email reminders.
- Generate a unique QR code for each product for quick lookup.
- Allow users to download PDF reports of their products and warranty status.
- Allow the admin to manage users and monitor overall platform activity.

2.3 User Classes

User Class	Responsibilities
User	Register/Login, add and manage products, upload bills and warranty cards, view service history, receive reminders, generate QR codes, download reports.
Admin	View and suspend user accounts, monitor overall system usage, view platform-wide statistics and reports.

2.4 Assumptions

- Users have internet connectivity and access to a modern web browser.
- Users provide accurate product and purchase information.
- A valid SMTP configuration is available for sending email reminders.

2.5 Constraints

- Authentication is mandatory for all core platform features.
- Requires a server running PHP 8 and MySQL (XAMPP environment for development).
- Development and testing occur on localhost; HTTPS enforcement applies only at production deployment.

3. Functional Requirements

FR1 — User Registration

- Users shall be able to register using name, email, phone number, and password.
- The system shall validate email format, phone number validity, and password strength.

FR2 — Login

- The system shall allow users to log in using email and password.
- The system shall authenticate users, maintain secure PHP sessions, and redirect them to role-specific dashboards (User or Admin).

FR3 — Product Management

- Users shall be able to add, update, delete, and view a list of their registered products.
- Product data includes: Product Name, Category, Brand, Purchase Date, Purchase Price, Seller/Store Name, and Product Image.

FR4 — Upload Bills

- Users shall be able to upload a scanned copy or photo of the purchase bill for each product.
- Uploaded bills shall be stored and linked to the corresponding product record.

FR5 — Upload Warranty Cards

- Users shall be able to upload the warranty card document/image for each product.
- Users shall enter warranty start date and warranty duration; the system shall compute the expiry date.

FR6 — Service History

- Users shall be able to add service records for a product, including service date, description, cost, and service center name.
- Users shall be able to view the complete service history of a product in chronological order.

FR7 — Warranty Reminder

- The system shall automatically check warranty and service due dates on a scheduled basis.
- The system shall flag products nearing warranty expiry (e.g., 30/15/7 days before) or with an upcoming service due date.

FR8 — Email Notification

- The system shall send email reminders to users via PHPMailer for upcoming warranty expiry and service due dates.

- Sent notifications shall be logged and viewable on the user's notification page.

FR9 — QR Code Generation

- The system shall generate a unique QR code for each registered product using PHP QR Code.
- Scanning the QR code shall open a read-only view of the product's key details and warranty status.

FR10 — Reports

- Users shall be able to generate and download a PDF report (using TCPDF) listing their products and warranty status.
- Reports shall be filterable by category and warranty status (active/expired).

FR11 — Admin Management

- Admins shall be able to view, search, and suspend user accounts.
- Admins shall be able to view platform-wide statistics such as total users, products, and active warranties.

4. Non-Functional Requirements

Security

- Passwords must be securely hashed using PHP password_hash() and verified with password_verify().
- HTTPS communication must be enforced at production deployment; local XAMPP development uses HTTP.
- Role-based access control (RBAC) to protect User and Admin-specific endpoints.
- PHP session timeouts for idle users.

Reliability

- System availability target: 99%.
- Daily MySQL database backups.

Performance

- Dashboard and search pages shall load within 3 seconds under normal usage.
- QR code generation and PDF report generation shall complete within 5 seconds.

Scalability

- The system architecture must support an increasing number of users and products.
- Modular PHP/MVC design to allow future enhancements without major redesign.

Maintainability

- Modular PHP architecture (MVC pattern) with clear documentation and code comments.

Usability

- Responsive UI design using Bootstrap 5 (mobile, tablet, desktop).
- Simple, intuitive dashboards with SweetAlert2 confirmations and DataTables listings.

5. External Interface Requirements

User Interface

The application shall contain the following primary views:

- Home Page
- Registration & Login Pages
- User Dashboard
- Add/Edit Product Page
- Product Details Page (bills, warranty card, service history, QR code)
- Service History Page
- Notifications Page
- Reports Page
- Admin Dashboard (user management, platform statistics)

Hardware Interface

Accessible via Laptop, Desktop, and Smartphone with a modern web browser.

Software Interface

- Frontend: HTML5, CSS3, JavaScript, Bootstrap 5
- Backend: PHP 8 (Core, MVC structure)
- Database: MySQL (hosted via XAMPP)
- Libraries: PHPMailer (email), TCPDF (reports), PHP QR Code (QR generation), Chart.js (dashboard charts), SweetAlert2 (alerts), DataTables (listings)

6. Database Design (Relational Schema for MySQL)

The tables below define the key fields required for the VaultKeep database. Data types and full constraints are determined during implementation.

Table: users

Field	Description
user_id	Primary key, auto-increment
name	Full name of the user
email	Unique email address, used for login
phone	Contact number
password	Hashed password
role	ENUM: user, admin
created_at	Account creation timestamp

Table: products

Field	Description
product_id	Primary key, auto-increment
user_id	Foreign key referencing users
product_name	Name of the product
category	Product category (e.g., electronics, appliance)
brand	Brand/manufacturer name
purchase_date	Date of purchase
purchase_price	Purchase amount
seller_name	Store/seller name
qr_code_path	Path to generated QR code image
created_at	Record creation timestamp

Table: warranties

Field	Description
warranty_id	Primary key, auto-increment
product_id	Foreign key referencing products
warranty_start_date	Start date of warranty period
warranty_duration_months	Warranty duration in months
warranty_expiry_date	Computed expiry date
status	ENUM: active, expiring_soon, expired

Table: documents

Field	Description
document_id	Primary key, auto-increment
product_id	Foreign key referencing products
document_type	ENUM: bill, warranty_card, product_image
file_path	Path to the stored file
uploaded_at	Upload timestamp

Table: service_history

Field	Description
service_id	Primary key, auto-increment
product_id	Foreign key referencing products
service_date	Date of service/repair
description	Details of the service performed
cost	Cost incurred
service_center	Name of the service center
next_service_due_date	Optional next due date

Table: notifications

Field	Description
notification_id	Primary key, auto-increment
user_id	Foreign key referencing users
product_id	Foreign key referencing products
type	ENUM: warranty_expiry, service_due
message	Notification text
is_read	Boolean flag
sent_at	Timestamp when the email was sent

Table: admins

Field	Description
admin_id	Primary key, references users(user_id)
last_login	Timestamp of last admin login

7. Use Case Summary

Use Case	Actor(s)	Description
Register / Login	User	Users create accounts and authenticate via PHP sessions.
Add Product	User	User registers a new product with purchase details.
Upload Bill / Warranty Card	User	User uploads supporting documents for a product.
Add Service Record	User	User logs a service/repair event for a product.
View Warranty Status	User	User views active, expiring, or expired warranties.
Receive Email Reminder	User	System sends an email before warranty/service due date.
Generate QR Code	User	System creates a QR code linked to product details.
Download Report	User	User downloads a PDF report of products and warranties.
Manage Users	Admin	Admin views or suspends user accounts.
Monitor Platform	Admin	Admin views platform-wide statistics.

8. Conclusion

VaultKeep aims to provide a simple, secure, and reliable solution for individuals to digitally organize their purchase bills, warranty cards, and service records. By leveraging a PHP and MySQL (XAMPP) stack along with lightweight libraries for email, PDF, and QR code generation, the system removes the risk of losing paper documents and helps users stay ahead of warranty and service deadlines through timely reminders.

9. Future Scope

- OCR-based automatic extraction of bill and warranty details from uploaded images.
- Native mobile application for Android and iOS.
- Cloud backup and sync of documents across devices.
- Barcode scanner integration for faster product entry.